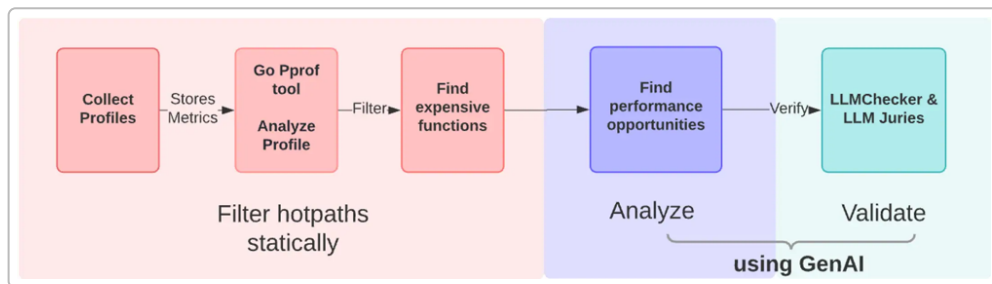


AI辅助软件性能缺陷分析综述

执行摘要

软件性能缺陷（performance bug/regression）指导致程序功能正确但执行效率显著下降的缺陷，往往难以通过传统测试发现^{① ②}。近年来，研究者尝试利用人工智能/机器学习方法自动化性能缺陷定位和根因分析。典型工作包括AutoPerf（NeurIPS 2019）采用硬件性能计数器与自编码器相结合的“零正样本学习”方法自动检测并定位软件性能回归^{③ ④}；Reveal（2026年）利用Linux **perf stat** 等低级硬件指标，并通过无监督学习（Z-Score、PCA、Isolation Forest等）进行异常检测以识别深度学习系统中的性能瓶颈^{⑤ ⑥}；工业界方面，Uber的PerfInsights系统结合全量运行时分析和大型语言模型（LLM），自动发现Go服务中的性能反模式并提出优化建议^{⑦ ⑧}。现有研究表明，融合性能计数器数据、程序调用信息与机器学习，可以在多种环境中有效地发现性能缺陷。然而，性能分析数据稀缺、根因定位难度大、系统复杂度高等挑战依然存在^{⑨ ⑩}。



图：Uber PerfInsights 性能优化流程。该工业案例通过收集运行时CPU/内存分析和性能反模式，实现热路径函数过滤和智能检测优化建议^{⑦ ⑧}。

学术研究进展

学术界已有多个研究致力于应用AI/ML方法进行性能缺陷检测与定位。在算法层面，Alam等人于2019年提出了**AutoPerf**系统：该系统使用硬件性能计数器收集程序在代表性输入下的执行数据，然后基于**自编码器**构建性能模型，通过“零正样本学习”检测性能回归^{③ ④}。具体来说，AutoPerf 将多线程程序划分为代码段，对每段在正常与回归版本下收集缓存未命中、指令数等**硬件计数器**（perf events）特征；然后训练自编码器，检测新版本执行时的异常代码段，从而定位回归的热函数。实验中，AutoPerf 在7个开源项目和benchmark上验证了10个真实性能缺陷，结果显示其能够准确发现所有缺陷（未漏报）且仅有微小的性能开销^{② ⑪}。

另一类研究关注静态代码特征。Zhao等人于2024年在MSR提出利用**性能代码度量**进行性能bug预测的方法^{⑫ ⑬}。他们定义了包括循环内条件判断、I/O操作数等在内的一系列静态代码指标，并结合传统的代码度量和开发过程特征，使用随机森林、XGBoost等模型在80个开源Java项目上进行性能bug预测^{⑬ ⑭}。结果显示，使用这些静态性能度量可以达到0.84左右的AUC、0.38的Matthews相关系数，显著优于仅用常规模式匹配的方法^{⑬ ⑭}。该工作提供了代码级别的性能缺陷预测思路，但由于只利用静态特征，难以给出运行时的根本原因函数定位。

此外，有学者从经验角度开展性能回归根因定位的研究。比如Liao等（2021）报道了在Web系统现场环境中基于统计模型的性能回归定位方法，但该方法主要是通过黑盒性能模型和统计分析定位有问题的系统活动，对应运行时指

标（如响应时间）与活动的关系建模¹⁵。总体而言，当前学术研究多侧重在模型架构与指标选取上，能够提供不同粒度的定位建议，但针对Linux perf等底层数据源的工作相对较少。

基于Linux perf的研究与工具

Linux perf 提供了丰富的硬件事件计数（如周期数、缓存命中）和调用栈采样功能，是性能分析的重要数据来源¹⁶。已有工作直接利用 perf 数据进行机器学习分析。例如，AutoPerf本质上就是将perf采集的计数器数据作为特征输入深度学习模型²。Intel/MIT团队还提出了类似思想的系统，通过自编码器和KMeans聚类对硬件计数器进行建模，用以诊断性能回归²¹¹。总体而言，此类方法依赖perf事件作为特征，最大优势是无需人为标注异常样本（零正样本学习），但也存在计数器信息难以直接映射到代码根因的局限⁹。

2025年发布的Reveal系统则是另一个典型例子。Reveal利用perf stat收集CPU、内存子系统等170余个低级指标，然后通过滑窗计算统计特征（均值、方差、时序特征等），再采用Z得分、主成分分析Mahalanobis距离与IsolationForest等无监督方法融合检测异常¹⁷¹⁸。该系统针对机器学习训练/推理任务，能够关联异常窗口到CPU、GPU、内存、网络等子系统瓶颈。Reveal目前计划开源代码和数据¹⁹，以推动社区研究。值得注意的是，基于perf的监控与分析也用于工业系统，例如一位Arm工程师演示了如何使用 perf record 和Intel SPE结合的Top-Down方法分析多线程程序的伪共享问题²⁰²¹。这些实践表明，将Linux perf与ML模型结合，可以在各种平台上发现低级硬件瓶颈，但也需要注意采样误差和归因偏差问题⁹。

工业实践与案例

工业界对AI辅助性能分析也非常关注。Intel研究者宣称AutoPerf系统已经在真是环境中测试过10个软件系统的性能回归，结果准确率高且无漏报¹¹²²。他们在博客中强调AutoPerf结合深度学习和硬件遥测，能够自动化地进行软件性能回归测试¹¹。此外，Uber于2025年发布了PerfInsights系统，这是一个生产级性能诊断平台²³²⁴。PerfInsights每天收集全站点Go服务的生产CPU/内存剖面数据，识别热函数（通常前30个CPU消耗最多的函数），并应用LLM对这些函数源代码进行分析匹配常见性能反模式²³⁸。该系统还通过“LLM陪审团”和规则检查等机制过滤虚假正例，实现高精度的优化建议。Uber报告称，该系统已经自动生成并合并了数百个性能优化提交，将开发者定位性能问题的时间缩短了90%以上²⁴。其他AI辅助工具/项目也在酝酿中，例如面向工程师的Perf-skills库收集了Linux perf 分析的AI助手技能（步骤建议），以及Windsurf IDE等集成AI性能分析辅助的尝试。这些工业实践表明，将机器学习/大模型与应用级或系统级性能数据结合，能显著提高性能分析效率，但通常依赖大量真实数据并需严密的验证机制。

方法比较

下面对代表性方法进行对比（表中“数据类型”指输入特征，“评估指标”指结果度量）：

方法/工具	输入数据	ML模型/算法	特征工程	评估指标	数据集	优势	限制	复现
AutoPerf (NeurIPS'19) 3 4	Linux perf硬件事件计数（cache miss, cycles等），针对关键函数采样的统计和调用栈数据	自编码器 + KMeans (零正样本学习)	函数级硬件计数器聚合	命中率、Top-K 精准度，Profiling开销等	7个程序共10个真实性能缺陷 2	无需正样本训练，可检测并定位并行程序性能回归；0%漏报 2	模型训练时间长（实验中高达5小时）；仅基于计数器，难以直接还原代码根因 9	开源复现代码+论文均25 2
Reveal (2026) 5 6	Linux perf 统计数（CPU利用率、内存吞吐等）+procfs+NVIDIA-smi等高维时序指标	无监督 (Z得分、PCA+Mahalanobis、Isolation Forest)	滑动窗统计特征 (均值、方差、自相关等) + 派生 IPC/ LLC 等指标	异常检测 F1，子系统定位准确率	多节点 GPU/ CPU ML 任务（LLM、CV模型等）	全天候监控，自动归因到CPU/GPU/内存/网络子系统；适应性强，无需预标记异常 18 6	当前采样率和窗口固定，对动态工作负载适应有限；检测结果需人工解释及干预 6	数据集和计划源 (1承)

方法/工具	输入数据	ML模型/算法	特征工程	评估指标	数据集	优势	限制	复现
PerfInsights (Uber 2025) 23 8	Go 服务的实时 CPU/内存剖面 (Profiler) + 完整函数源代码 + 性能反模式库	大型语言模型 (GPT 类) + 规则验证 (LLMCheck)	提取热门函数及其源代码，将其与已知反模式列表共同输入 LLM	合并优化建议数量/质量，开发者采纳率	Uber内部数百个性能问题用例	自动化程度高，结合AI和知识库实现可解释优化；已显著降低人工调优成本 24	依赖大规模公司级 Profiler 和定制 LLM；结果基于经验规则，难以推广到完全不同环境	内部系统，公开实现 (Uber 博客描述，方法论)
PerfBench (Perf. Bug Benchmark) 1 10	.NET应用的性能测试结果与相关代码改动	无 (评估平台)	基于对比基准的性能度量	AGENT修复成功率	81个真实性能缺陷修复用例 26	提供公开性能缺陷修复基准，评估 AI代码代理修复能力；验证了现有模型在性能优化任务上表现低 (3%-20%) 10	侧重修复自动化，不涉及根因分析本身；基准主要包含 .NET 样本，其他语境泛化受限	开源，数据集基准代码布

方法/工具	输入数据	ML模型/算法	特征工程	评估指标	数据集	优势	限制	复现
Zhao et al. MSR'24 ¹² ¹³	代码静态度量 (循环数、分支数、I/O操作数等)、变更过程特征等	RF, XGBoost, LR 等多种模型	静态代码性能相关度量	AUC≈0.84, PR-AUC≈0.21, MCC≈0.38 ¹³	80个开源Java项目性能缺陷数据集	静态分析无需运行时数据, 可提前预测项目中可能出现的性能bug; 提供了复现包 ¹³ ¹⁴	精度依赖于特征设计; 仅能到文件级别预测, 无法给出具体的函数/运行时上下文; 忽略运行时硬件信息	复现 提 可 数 代 ¹³ ¹⁴

实践工具链与 workflow

将Linux perf与机器学习结合进行性能分析，通常包含**数据收集 - 特征提取 - 模型训练/推理 - 根因定位**几步。一个典型 workflow 如下：首先使用命令行工具如 `perf stat` 或 `perf record` 采集目标程序在代表输入下的硬件计数器数据与调用栈。例如：

```
perf stat -r3 -d ./myapp      # 收集整体统计数据（周期、指令数、缓存访问等）
sudo perf record -F 99 -g -e cycles,instructions -o perf.data ./myapp # 收集事件
                               样本和调用图
perf script -i perf.data --fields symbol > callstacks.txt           # 导出调用栈
                               列表
```

然后，利用脚本或Python程序解析perf数据并构建机器学习输入特征。例如，可将每个采样中的硬件事件计数聚合为函数级特征，也可计算时间序列统计（滑动均值、方差等）²⁵ ¹⁷。接着，将处理好的特征输入ML模型进行训练或预测。模型类型可以是**自编码器**用于无监督异常检测³，也可以是**随机森林/XGBoost**用于缺陷预测¹³，甚至是**大型语言模型**用于代码反模式分析⁸。最后，根据模型输出定位可疑的函数或子系统，并生成诊断报告。例如，AutoPerf将检测到的异常函数标记出来²；Reveal会将异常窗口归因到CPU或GPU子系统¹⁸；PerfInsights则直接生成可执行优化补丁建议⁸ ²⁴。下图用流程图示意该 workflow：



在实践中，各种开源工具也可集成到此流程中。例如，可使用 `perf-tools` 和 `perf script` 导出数据，用 Python 的 `scikit-learn` 或 `PyTorch` 等库构建模型；也可借助商业监控平台（如 `Datadog`、`Dynatrace`）自动汇总指标并触发 ML 分析。Workflow 通常需要结合业务场景灵活设计，如在 CI 流水线中自动化回归测试、在生产环境实时监控并报警等。

挑战与未来方向

尽管已有研究和实践取得进展，但 AI 辅助性能分析仍面临诸多挑战：首先，**缺乏标注数据**。真实性能缺陷案例稀少，难以获得充足的监督学习样本。现有性能缺陷基准（如 `PerfBench` ²⁶）规模有限，多集中于特定语言或框架；而如 `AutoPerf`、`Reveal` 等方法更多依赖无监督或半监督学习来缓解此问题 ² ¹⁸。其次，**根因定位困难**。硬件计数器和低层性能指标往往只能指出“哪里出了问题”（如哪个函数耗时异常或哪个子系统出现瓶颈），但难以明确指出**为什么**和**如何修复**。例如，`AutoPerf` 作者也指出仅凭 HWPC 计数器信息很难直接还原到具体代码缺陷 ⁹。另外，**系统复杂性和噪声**也是难点：在分布式或云平台上，工作负载和外部干扰多变，单节点采样的数据可能不稳定。如何在持续运行的生产环境中实时检测并自动化定位，是未来重点。`Reveal` 团队提到当前采用固定采样率和窗口，未来将考虑**动态调整采样**和**自动干预**（如自动调整 CPU 亲和性）⁶。

未来研究方向包括：(1) **多模态特征融合**，结合日志、调用跟踪、网络流量等多种数据源，提升诊断准确度；(2) **大模型与知识库结合**，如 `Uber` 的做法，用 LLM 结合领域规则自动给出修复建议 ⁸ ²⁴；(3) **可解释 AI**，增强模型透明性，让工程师更易理解根因；(4) **自动化闭环**，开发能够自动定位并轻量级修复性能缺陷（例如自动调整参数或插桩）的工具；(5) **公共基准与开源**，如 `PerfBench` 等工作为社区提供了宝贵数据，后续需要更多公开数据集和基准，促进算法评测和复现。正如 `Intel` 研究所述，性能回归检测关系到软件质量与业务成本 ¹¹，使用 AI 进行自动化诊断虽已有可喜成果，但仍需在算法泛化、工程实践和生态建设上持续探索。

参考文献：（文中引用格式【*tl-L**】对应附录所示行号）

¹ ¹⁰ ²⁶ `PerfBench: Can Agents Resolve Real-World Performance Bugs?`

<https://arxiv.org/html/2509.24091v2>

² ³ ⁴ ⁹ `A Zero-Positive Learning Approach for Diagnosing Software Performance Regressions`

<https://people.csail.mit.edu/tatbul/publications/NeurIPS19.pdf>

⁵ ⁶ ¹⁷ ¹⁸ ¹⁹ `Detecting Anomalies in Systems for AI Using Hardware Telemetry`

<https://arxiv.org/html/2510.26008v2>

⁷ ⁸ ²³ `PerfInsights: Detecting Performance Optimization Opportunities in Go Code using Generative AI`

<https://www.uber.com/us/en/blog/perfinsights/>

¹¹ ²² `Three Pillars of Machine Programming Provide Core Concepts for...`

<https://www.intel.com/content/www/us/en/research/blogs/three-pillars-of-machine-programming-research-advances.html>

¹² ¹³ ¹⁴ `Enhancing Performance Bug Prediction Using Performance Code Metrics`

https://safwathassan.com/publications/Performance_Bug_Prediction_MSR.pdf

¹⁵ `hengli.org`

<https://www.hengli.org/pdf/Liao2021LocatePerfReg.pdf>

¹⁶ `usenix.org`

<https://www.usenix.org/system/files/osdi25-li.pdf>

20 21 Perform root cause analysis with Perf C2C | Arm Learning Paths

<https://learn.arm.com/learning-paths/servers-and-cloud-computing/false-sharing-arm-spe/how-to-4/>

24 LLMOps in Production: Another 419 Case Studies of What Actually Works - ZenML Blog

<https://www.zenml.io/blog/llmops-in-production-another-419-case-studies-of-what-actually-works>

25 GitHub - mejbah/AutoPerf · GitHub

<https://github.com/mejbah/AutoPerf>